

Núcleo 3 — Previsão de Risco de Frustração/Fadiga

Aluna: Maria Clara Bragança

Disciplina: Data Mining

Metodologia: CRISP-DM

Fase 1 — Business Understanding

1. Contexto e Problema de Negócio

As análises anteriores extraíram propriedades do texto de uma atividade (complexidade, intenção pedagógica, demanda psicomotora) e construíram um Grafo de Conhecimento que representa o domínio de cada aluno sobre conceitos específicos, a partir de sinais de acerto e latência. Falta uma peça complementar: um sinal de alerta precoce. Se um aluno está encadeando tentativas mal sucedidas, o risco de frustração e fadiga aumenta antes que o desempenho colapse — identificar esse padrão a tempo permite adaptar a atividade (reduzir a dificuldade, trocar o formato, sugerir uma pausa) antes que o aluno abandone a tarefa.

2. Objetivo de Negócio

Avaliar se é possível prever, a partir do histórico recente de um aluno em uma sequência de exercícios, se a próxima tentativa tem alta probabilidade de erro — um proxy comportamental de risco de frustração, já que uma sequência de erros consecutivos é um precursor observável de desengajamento, mesmo sem medir o estado afetivo diretamente (esse último é papel da pulseira biométrica do sistema, um sinal diferente e complementar, não substituído por esta análise). Como deixar passar um aluno em risco tem custo maior do que um alarme falso ocasional (a janela de intervenção se fecha sem que o sistema tenha percebido), o critério de sucesso não é a acurácia geral do modelo, e sim sua capacidade de identificar a maior parte dos casos de risco real (recall da classe de risco).

3. Objetivo da Mineração de Dados

Construir, a partir do log de interações do ASSISTments, um conjunto de features que descreva o histórico de cada aluno até o momento imediatamente anterior a cada tentativa (sem usar informação da própria tentativa, o que caracterizaria vazamento de dado), treinar e comparar classificadores binários para prever erro na tentativa seguinte, corrigir o modelo escolhido para priorizar recall da classe de risco, e explicar as previsões com SHAP — de forma legível tanto para o professor (linguagem técnica) quanto para os responsáveis (linguagem acessível).

4. Perguntas Estratégicas

- O histórico recente de acerto e tentativas de um aluno é suficiente para prever a próxima tentativa com uma acurácia relevante acima da linha de base?
 - Um modelo não linear (XGBoost) supera uma linha de base simples nesse problema, e supera também outros algoritmos candidatos (Regressão Logística, Random Forest) sob as mesmas condições?
 - Um modelo otimizado para acurácia atende ao objetivo de negócio de sinalizar risco com antecedência, ou é necessário corrigir o treinamento para priorizar recall da classe de risco?
 - Quais variáveis mais influenciam a previsão, e essa influência é interpretável em termos pedagógicos?
-

Fase 2 — Data Understanding

Por que processar com Spark nesta análise

As análises anteriores trabalharam com o ASSISTments já agregado por aluno e habilidade (uma linha por combinação aluno-habilidade). Esta análise precisa da granularidade oposta: uma linha por interação individual, em ordem cronológica, para calcular o histórico de cada aluno até o momento imediatamente anterior a cada tentativa. Isso significa uma função de janela (*window function*) — média móvel de acerto por aluno, respeitando a ordem das interações — sobre as 274 mil interações individuais do dataset. É exatamente o tipo de operação para o qual o Spark existe: agregações particionadas e ordenadas em paralelo, sobre um volume de dado que já não é trivial de inspecionar linha a linha. Uso o Spark aqui, não pandas, por esse motivo específico — não é uma escolha arbitrária de ferramenta.

Importação e carregamento

```
In [1]: # Eu importo o SparkSession, ponto de entrada para qualquer operacao no Spark
from pyspark.sql import SparkSession, Window
# Eu importo as funcoes de coluna do Spark, usadas para as transformacoes abaixo
from pyspark.sql import functions as F

# Eu crio a sessao Spark Local (roda no proprio computador, sem cluster externo)
spark = (
    SparkSession.builder
        .appName("nucleo3_risco_frustracao")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "4")
        .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")

print(f"Sessao Spark iniciada - versao {spark.version}")
```

```
In [2]: # Eu carrego o mesmo dataset ASSISTments 2009 usado na análise anterior, agora
via Spark
DATASETS = "../../datasets/raw"
assistments = spark.read.parquet(f"
{DATASETS}/assistments2009/assistments2009_train.parquet")

print(f"Total de alunos: {assistments.count()}")
assistments.printSchema()
```

Total de alunos: 4148

```
root
|-- user_id: long (nullable = true)
|-- skill_ids: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- skill_names: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- grades: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- attempt_counts: array (nullable = true)
|   |-- element: long (containsNull = true)
|-- answer_types: array (nullable = true)
|   |-- element: string (containsNull = true)
```

Interpretação

O schema confirma as mesmas colunas já conhecidas da análise anterior: `user_id` e quatro colunas do tipo lista (`skill_names` , `grades` , `attempt_counts` , `answer_types`), uma lista por aluno contendo toda a sequência de interações dele. A tarefa desta fase é transformar essas listas em uma tabela de interações individuais, uma linha por tentativa, preservando a ordem — é essa ordem que permite calcular o histórico sem vazamento na fase seguinte.

Fase 3 — Data Preparation

Explosão das listas em interações individuais

Cada aluno tem uma lista de interações; preciso de uma linha por interação, com a posição dela na sequência do aluno preservada. Uso `arrays_zip` para combinar as quatro listas em uma lista de estruturas (uma struct por interação), e depois `explode` para transformar cada elemento dessa lista em uma linha da tabela — a função nativa do Spark para esse tipo de transformação, equivalente ao que fiz manualmente em pandas na análise anterior, mas expressando a operação de forma distribuída.

```
In [3]: # Eu combino as quatro listas paralelas em uma unica lista de structs (uma
struct por interacao)
interacoes_zipadas = assistments.withColumn(
    "interacao",
    F.arrays_zip("skill_names", "grades", "attempt_counts", "answer_types"),
)
```

```
# Eu exploto a lista de structs: cada elemento vira uma linha, mantendo o
user_id
interacoes = interacoes_zipadas.select(
    "user_id",
    F.posexplode("interacao").alias("posicao", "interacao_struct"),
).select(
    "user_id",
    "posicao",
    F.col("interacao_struct.skill_names").alias("skill"),
    F.col("interacao_struct.grades").cast("int").alias("acerto"),

    F.col("interacao_struct.attempt_counts").cast("double").alias("tentativas"),
    F.col("interacao_struct.answer_types").alias("tipo_resposta"),
)

print(f"Total de interacoes individuais: {interacoes.count()}")
interacoes.show(5)
```

Total de interacoes individuais: 274331

user_id	posicao	skill	acerto	tentativas	tipo_resposta
14	0	Circle Graph	0	1.0	algebra
14	1	Circle Graph	1	1.0	algebra
14	2	Circle Graph	0	1.0	algebra
14	3	Circle Graph	0	1.0	algebra
14	4	Circle Graph	0	1.0	algebra

only showing top 5 rows

Interpretação

O total de interações bate com o valor já obtido na análise anterior (274.331), confirmando que a explosão via Spark preservou todas as linhas — nenhuma interação foi perdida ou duplicada na transformação. A coluna `posicao` registra a ordem original de cada interação dentro da sequência do aluno (0-indexada), essencial para a próxima etapa.

Features históricas via função de janela (sem vazamento)

Para cada interação, calculo o histórico do aluno **considerando apenas as interações anteriores a ela** — nunca a interação atual, o que seria vazamento de dado (a variável que estou tentando prever entraria como feature). A função de janela do Spark (`Window.partitionBy("user_id").orderBy("posicao")`) particiona o dado por aluno, ordena pela posição, e `rowsBetween(Window.unboundedPreceding, -1)` restringe o cálculo a tudo que veio antes da linha atual, excluindo-a — essa é a garantia estrutural contra vazamento, imposta pela própria definição da janela, não por uma verificação manual posterior.

Depois de validar que versões anteriores desta análise deixavam sinal disponível sem uso (algoritmos diferentes convergindo para o mesmo teto de desempenho é evidência de que o gargalo está nas features, não no modelo), adiciono features novas, todas derivadas das mesmas colunas já disponíveis no dataset, sem precisar de dado novo:

- **Taxa de acerto em janela recente** (últimas 5 tentativas, `rowsBetween(-5, -1)`), em vez de só a taxa acumulada desde o início da sequência — a acumulada dilui uma queda de desempenho recente.
- **Sequência de erros consecutivos** imediatamente antes da tentativa atual — um proxy mais direto de frustração do que uma média, já que é justamente uma sequência de erros seguidos (não a taxa de acerto isolada) que caracteriza o precursor comportamental definido na Fase 1.
- **Taxa de acerto do aluno na mesma habilidade específica** (não a média geral) — um aluno pode estar indo bem em geral mas mal numa habilidade puntual, ou o contrário; essa feature separa desempenho geral de desempenho específico.
- **Tipo de resposta esperado** (`tipo_resposta` — `algebra`, `choose_1`, `fill_in_1`, `choose_n`, `open_response`), até aqui completamente ignorado apesar de disponível no dataset — o formato da resposta esperada pode correlacionar com a dificuldade percebida pelo aluno.

Numa rodada seguinte, motivada por uma pergunta direta sobre até onde dava para espremer sinal do mesmo dataset, acrescento mais três sinais:

- **Volatilidade recente** (desvio padrão do acerto nas últimas 5 tentativas) — duas taxas de acerto recentes iguais podem esconder padrões de risco diferentes (um aluno estável vs. um alternando acerto e erro).
- **Número de tentativas anteriores na mesma habilidade** (`n_tentativas_habilidade_anteriores`) — informa ao modelo o quão confiável é a estimativa de `taxa_acerto_na_habilidade`; poucas tentativas anteriores tornam essa taxa instável, o mesmo raciocínio já aplicado ao filtro de histórico mínimo.
- Retomo `posicao` (quantos exercícios o aluno já fez na sequência) como feature de fato — ela já estava disponível desde a primeira versão desta análise, mas tinha ficado de fora da lista de features por um esquecimento na hora de montar a lista de colunas do modelo; é um proxy direto de fadiga acumulada na sessão, sem vazamento (é conhecida antes da tentativa).

Uma quarta feature nova — a dificuldade média da habilidade entre todos os alunos — só pode ser calculada depois da separação treino/teste (Fase 4), por um motivo de vazamento diferente dos anteriores; a explicação está lá.

Mantenho a versão acumulada e a versão recente da taxa de acerto como features separadas, em vez de substituir uma pela outra, e deixo o próprio modelo (e a análise SHAP na Fase 6) indicar qual carrega mais informação.

```
In [4]: # Eu defino a janela acumulada: particionada por aluno, ordenada pela posicao,
# olhando so para tras (do inicio da sequencia ate a linha anterior, nunca a
# atual)
janela_historico = (
    Window.partitionBy("user_id")
    .orderBy("posicao")
    .rowsBetween(Window.unboundedPreceding, -1)
)
```

```

# Eu defino a janela recente: mesma logica, mas limitada as ultimas 5
interacoes anteriores
JANELA_RECENTE_TAMANHO = 5
janela_recente = (
    Window.partitionBy("user_id")
        .orderBy("posicao")
        .rowsBetween(-JANELA_RECENTE_TAMANHO, -1)
)

# Eu defino a janela historica especifica por habilidade: mesma logica da
acumulada,
# mas particionada tambem por skill – mede o desempenho do aluno so naquela
habilidade
janela_habilidade = (
    Window.partitionBy("user_id", "skill")
        .orderBy("posicao")
        .rowsBetween(Window.unboundedPreceding, -1)
)

# Eu defino a janela "ate a linha atual" (inclusive), usada so como etapa
# intermediaria para calcular a sequencia de erros consecutivos abaixo
janela_ate_atual =
Window.partitionBy("user_id").orderBy("posicao").rowsBetween(Window.unboundedPr
ecedding, 0)
janela_por_aluno = Window.partitionBy("user_id").orderBy("posicao")

interacoes_com_historico = (
    interacoes
    # Eu conto quantas interacoes anteriores o aluno teve, para poder filtrar
historico insuficiente depois
    .withColumn("n_interacoes_anteriores",
F.count("acerto").over(janela_historico))
    # Eu calculo a taxa de acerto historica acumulada (media do acerto desde o
inicio da sequencia)
    .withColumn("taxa_acerto_historica",
F.avg("acerto").over(janela_historico))
    # Eu calculo a taxa de acerto na janela recente (media das ultimas 5
interacoes anteriores)
    .withColumn("taxa_acerto_recente", F.avg("acerto").over(janela_recente))
    # Eu calculo a volatilidade recente: desvio padrao do acerto nas ultimas 5
interacoes. Duas taxas de acerto recentes iguais podem esconder padroes
# bem diferentes (uma sequencia estavel vs. uma alternando acerto/erro) –
# a volatilidade distingue os dois casos
    .withColumn("volatilidade_recente",
F.stddev(F.col("acerto")).over(janela_recente))
    # Eu calculo a media de tentativas historica, mesma logica da acumulada
    .withColumn("tentativas_media_historica",
F.avg("tentativas").over(janela_historico))
    # Eu calculo a media de tentativas na janela recente, mesma logica da taxa
de acerto recente
    .withColumn("tentativas_media_recente",
F.avg("tentativas").over(janela_recente))
    # Eu calculo a taxa de acerto do aluno especificamente nessa habilidade
(nao a media geral)
    .withColumn("taxa_acerto_na_habilidade",
F.avg("acerto").over(janela_habilidade))
    .withColumn("n_tentativas_habilidade_anteriores",
F.count("acerto").over(janela_habilidade))
    # --- sequencia de erros consecutivos: tecnica de "grupo por run" ---
    # grp incrementa (soma cumulativa inclusiva) a cada acerto=1; entre dois

```

```

# acertos, grp fica constante, entao toda a sequencia de erros consecutivos
# cai no mesmo grupo. row_number() dentro do grupo da a posicao da linha
# dentro dessa sequencia; subtraio 1 para descontar a linha de acerto que
# abriu o grupo (exceto no grupo 0, que nao tem linha de acerto nenhuma –
# é o caso do aluno cujas primeiras interacoes registradas ja sao erros)
.withColumn("grupo_streak", F.sum(F.col("acerto")).over(janela_ate_atual))
.withColumn(
  "posicao_no_grupo",
  F.row_number().over(Window.partitionBy("user_id",
"grupo_streak").orderBy("posicao")),
)
.withColumn(
  "sequencia_erros_atual",
  F.when(F.col("grupo_streak") == 0, F.col("posicao_no_grupo"))
  .otherwise(F.col("posicao_no_grupo") - 1),
)
# a feature usada no modelo e' a sequencia de erros ATE A LINHA ANTERIOR
# (lag), nunca incluindo a linha atual – mesma garantia contra vazamento
.withColumn(
  "sequencia_erros_anterior",
  F.coalesce(F.lag("sequencia_erros_atual", 1).over(janela_por_aluno),
F.lit(0)),
)
.drop("grupo_streak", "posicao_no_grupo", "sequencia_erros_atual")
)

interacoes_com_historico.select(
  "user_id", "posicao", "acerto", "n_interacoes_anteriores",
  "taxa_acerto_recente", "volatilidade_recente", "sequencia_erros_anterior",
  "taxa_acerto_na_habilidade", "n_tentativas_habilidade_anteriores",
).show(10)

```

```

+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+
|user_id|posicao|acerto|n_interacoes_anteriores|taxa_acerto_recente|volatilidade_
recente|sequencia_erros_anterior|taxa_acerto_na_habilidade|n_tentativas_habilidad
e_anteriores|
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+
| 54318|      0|      0|              0|          NULL|          NULL|
NULL|              0|          NULL|
0|
| 54318|      1|      1|              1|          1|          0.0|
NULL|              1|          0.0|
1|
| 54318|      2|      1|              2|          0.5|          0.5|
0.7071067811865476|              0|
2|
| 54318|      3|      1|              3| 0.6666666666666666|
0.5773502691896258|              0| 0.6666666666666666|
3|
| 54318|      4|      1|              4|          0.75|          0.75|
0.5|              0|
4|
| 54318|      5|      0|              5|          0.8|          0.8|
0.447213595499958|              0|
5|
| 54318|      6|      1|              6|          0.8|          0.8|
0.4472135954999579|              1| 0.6666666666666666|
6|
| 54318|      7|      1|              7|          0.8|          0.8|
0.4472135954999579|              0| 0.7142857142857143|
7|
| 54318|      8|      1|              8|          0.8|          0.75|
0.4472135954999579|              0|
8|
| 54318|      9|      1|              9|          0.8|          0.8|
0.447213595499958|              0| 0.7777777777777778|
9|
+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+
only showing top 10 rows

```

Interpretação

A tabela confirma a lógica esperada das novas features: `sequencia_erros_anterior` sobe a cada erro consecutivo e volta a zero assim que um acerto ocorre.

`taxa_acerto_na_habilidade` coincide com a taxa recente/histórica neste trecho porque, nas primeiras interações do aluno, ele ainda não repetiu nenhuma habilidade — as colunas só divergem quando o aluno alterna entre habilidades diferentes.

`n_tentativas_habilidade_anteriores` mostra a cobertura crescente do histórico por habilidade, usada para decidir quando aplicar o valor de reserva (*fallback*) na Fase 3, e depois reaparece como feature própria — o SHAP na Fase 6 mostra que essa contagem carrega sinal preditivo por si só, não é só um auxiliar de outra feature.

In [5]: MINIMO_HISTORICO = 5

```
# Eu mantenho so interacoes com pelo menos 5 interacoes anteriores validas,
# pelo mesmo motivo de confiabilidade estatistica ja discutido na analise
anterior
base_filtrada =
interacoes_com_historico.filter(F.col("n_interacoes_anteriores") >=
MINIMO_HISTORICO)

# Eu identifico as 10 habilidades mais frequentes, para usar como variavel
categorica
# (codificar todas as centenas de habilidades como colunas separadas nao seria
viavel)
top_skills = (
    base_filtrada.groupBy("skill").count()
    .orderBy(F.desc("count"))
    .limit(10)
    .select("skill")
    .rdd.flatMap(lambda linha: linha)
    .collect()
)

base_com_skill_top = (
    base_filtrada
    .withColumn(
        "skill_agrupada",
        F.when(F.col("skill").isin(top_skills),
F.col("skill")).otherwise(F.lit("outra")),
    )
    # Eu preencho a taxa de acerto na habilidade especifica com a taxa
acumulada
    # geral quando o aluno ainda nao tem nenhuma tentativa anterior naquela
# habilidade (10,9% dos casos, medido na exploracao) – fallback razoavel,
# ja que e' a melhor estimativa disponivel na ausencia de dado especifico
    .withColumn(
        "taxa_acerto_na_habilidade",
        F.coalesce(F.col("taxa_acerto_na_habilidade"),
F.col("taxa_acerto_historica")),
    )
    # Eu calculo a tendencia de desempenho: recente menos acumulada. Positivo
# indica melhora recente, negativo indica deterioracao – a informacao de
# direcao fica explicita numa unica feature, em vez de depender do modelo
# aprender essa combinacao a partir das duas features separadas (util
# sobretudo para o modelo linear, que nao capta interacoes automaticamente)
    .withColumn(
        "tendencia_desempenho",
        F.col("taxa_acerto_recente") - F.col("taxa_acerto_historica"),
    )
)

print(f"Interacoes apos filtro de historico minimo:
{base_com_skill_top.count()}")
print(f"Habilidades mantidas individualmente: {top_skills}")
```

Interacoes apos filtro de historico minimo: 255003

Habilidades mantidas individualmente: ['Equation Solving Two or Fewer Steps', 'Conversion of Fraction Decimals Percents', 'Addition and Subtraction Integers', 'Addition and Subtraction Fractions', 'Equation Solving More Than Two Steps', 'Proportion', 'Subtraction Whole Numbers', 'Multiplication and Division Integers', 'Pythagorean Theorem', 'Table']

```
In [6]: # Eu trago o resultado processado no Spark para o pandas – a partir daqui,
# o volume de dado (algumas dezenas de milhares de linhas) e pequeno o
# suficiente para as bibliotecas de modelagem, que nao sao distribuidas
base_pd = base_com_skill_top.select(
    "user_id", "posicao", "acerto",
    "taxa_acerto_historica", "taxa_acerto_recente", "volatilidade_recente",
    "tendencia_desempenho",
    "tentativas_media_historica", "tentativas_media_recente",
    "taxa_acerto_na_habilidade", "n_tentativas_habilidade_anteriores",
    "sequencia_erros_anterior",
    "skill_agrupada", "tipo_resposta",
).toPandas()

# Eu converto o tipo de resposta em colunas indicadoras (one-hot encoding).
# skill_agrupada NAO vira one-hot aqui – fica guardada como coluna bruta,
# usada na Fase 4 para calcular a dificuldade media da habilidade so' com
# dados do treino (ver explicacao la); as colunas indicadoras da habilidade
# ja existiam desde a rodada anterior e continuam sendo criadas mais abaixo,
# na propria Fase 4, junto com a codificacao segura da dificuldade
base_pd = base_pd.join(base_pd["tipo_resposta"].str.get_dummies())
base_pd = base_pd.join(base_pd["skill_agrupada"].str.get_dummies(),
rsuffix="_habilidade")
base_pd = base_pd.drop(columns=["tipo_resposta"])

print(f"Formato final: {base_pd.shape}")
base_pd.head()
```

Formato final: (255003, 29)

Out[6]:

	user_id	posicao	acerto	taxa_acerto_historica	taxa_acerto_recente	volatilidade_recer
0	54318	5	0	0.800000	0.8	0.4472
1	54318	6	1	0.666667	0.8	0.4472
2	54318	7	1	0.714286	0.8	0.4472
3	54318	8	1	0.750000	0.8	0.4472
4	54318	9	1	0.777778	0.8	0.4472

5 rows × 29 columns

Interpretação

A conversão para pandas acontece só depois de todo o processamento pesado (explosão das listas e cálculo das médias móveis) já ter sido feito no Spark — é esse

processamento, não o treino do modelo em si, que se beneficiaria de paralelização em um volume de dado maior. As bibliotecas de modelagem usadas a seguir (scikit-learn, XGBoost) trabalham sobre estruturas em memória, não distribuídas, o que é adequado para o tamanho da base após o filtro de histórico mínimo.

Fase 4 — Modelagem

Separação treino/teste estratificada e agrupada por aluno

A variável-alvo (`acerto` — 1 quando o aluno acerta a tentativa, 0 quando erra) tem uma distribuição desbalanceada, como é comum em dados educacionais. Uso separação **estratificada**: a proporção de acertos e erros é preservada tanto no conjunto de treino quanto no de teste, evitando que uma divisão aleatória concentre por acaso mais erros de um lado do que do outro.

Além da estratificação, a separação também precisa ser **agrupada por aluno** (`user_id`): cada aluno tem várias interações na base, e se uma divisão aleatória simples por linha permitir que interações do mesmo aluno caiam tanto no treino quanto no teste, o modelo deixa de ser avaliado quanto à sua capacidade de generalizar para um aluno nunca visto — ele pode estar, em parte, apenas reconhecendo o padrão de comportamento de um aluno específico que também apareceu no treino. Uso `StratifiedGroupKFold`, que combina as duas exigências (grupos inteiros de cada lado, proporção de classes preservada o quanto a restrição de grupo permitir), e tomo a primeira dobra como a divisão treino/teste.

```
In [7]: import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold, cross_val_score
from sklearn.dummy import DummyClassifier
from xgboost import XGBClassifier

FEATURES = [c for c in base_pd.columns if c not in ("user_id", "acerto",
"skill_agrupada")]
X = base_pd[FEATURES]
y = base_pd["acerto"]
grupos = base_pd["user_id"]
skill_bruta = base_pd["skill_agrupada"] # mantida a parte, usada so no
encoding seguro a seguir

print(f"Taxa de acerto geral (proporcao da classe majoritaria):
{y.mean():.1%}")
print(f"Total de alunos distintos: {grupos.nunique()}")
print(f"Features antes da codificacao de dificuldade da habilidade:
{FEATURES}")

# Eu uso StratifiedGroupKFold para obter a divisao treino/teste: nenhum aluno
# aparece nos dois lados, e a proporcao de acerto/erro fica preservada o quanto
# a restricao de grupo permitir. Tomo a primeira dobra (80% treino, 20% teste)
# como a divisao fixa usada no restante da analise.
validacao_estratificada = StratifiedGroupKFold(n_splits=5, shuffle=True,
random_state=42)
```

```

indice_treino, indice_teste = next(validacao_estratificada.split(X, y,
groups=grupos))

X_train, X_test = X.iloc[indice_treino].copy(), X.iloc[indice_teste].copy()
y_train, y_test = y.iloc[indice_treino], y.iloc[indice_teste]
grupos_train = grupos.iloc[indice_treino]
skill_bruta_treino = skill_bruta.iloc[indice_treino]
skill_bruta_teste = skill_bruta.iloc[indice_teste]

alunos_treino = set(grupos_train)
alunos_teste = set(grupos.iloc[indice_teste])
print(f"\nTreino: {len(X_train)} linhas, {len(alunos_treino)} alunos")
print(f"Teste: {len(X_test)} linhas, {len(alunos_teste)} alunos")
print(f"Alunos presentes nos dois conjuntos (deve ser 0): {len(alunos_treino &
alunos_teste)}")

```

Taxa de acerto geral (proporcao da classe majoritaria): 66.3%

Total de alunos distintos: 3452

Features antes da codificacao de dificuldade da habilidade: ['posicao', 'taxa_acerto_historica', 'taxa_acerto_recente', 'volatilidade_recente', 'tendencia_desempenho', 'tentativas_media_historica', 'tentativas_media_recente', 'taxa_acerto_na_habilidade', 'n_tentativas_habilidade_anteriores', 'sequencia_erros_anterior', 'algebra', 'choose_1', 'choose_n', 'fill_in_1', 'open_response', 'Addition and Subtraction Fractions', 'Addition and Subtraction Integers', 'Conversion of Fraction Decimals Percents', 'Equation Solving More Than Two Steps', 'Equation Solving Two or Fewer Steps', 'Multiplication and Division Integers', 'Proportion', 'Pythagorean Theorem', 'Subtraction Whole Numbers', 'Table', 'outra']

Treino: 215830 linhas, 2820 alunos

Teste: 39173 linhas, 632 alunos

Alunos presentes nos dois conjuntos (deve ser 0): 0

Dificuldade média da habilidade — codificação segura contra vazamento

Até aqui, a dificuldade de uma habilidade só aparece indiretamente, através da variável categórica (`skill_agrupada` , codificada em colunas indicadoras). Uma feature mais direta é a taxa de acerto **média de todos os alunos** naquela habilidade — uma medida de o quão difícil a habilidade é objetivamente, complementar à taxa de acerto do próprio aluno naquela habilidade (`taxa_acerto_na_habilidade` , que mede desempenho individual, não dificuldade populacional).

Essa feature precisa de um cuidado que as anteriores não precisavam: ela é calculada a partir da própria variável-alvo (`acerto`), então, se for calculada sobre a base inteira antes da divisão treino/teste, informação do conjunto de teste vazaria para dentro do treino de forma indireta — o modelo estaria, em parte, "vendo" o resultado médio de exemplos que deveria prever sem ter visto. Por isso, calculo a média **só com dados do conjunto de treino** (nunca do teste) e aplico esse mesmo valor, fixo, também ao conjunto de teste — a mesma lógica de um `fit` que só acontece no treino e um `transform` aplicado aos dois lados, comum em codificadores do scikit-learn, embora aqui feito manualmente. Habilidades presentes no teste que não aparecerem no treino (não esperado, já que a categorização por `skill_agrupada` foi feita sobre a base inteira antes do split, mas verificado por segurança) recebem a média geral do treino como valor de reserva.

```
In [8]: # Eu calculo a dificuldade da habilidade (1 - taxa media de acerto) usando
# APENAS o conjunto de treino, para nao vazar informacao do teste
media_acerto_habilidade_treino = y_train.groupby(skill_bruta_treino).mean()
media_acerto_geral_treino = y_train.mean()

dificuldade_por_habilidade = 1 - media_acerto_habilidade_treino
dificuldade_geral_treino = 1 - media_acerto_geral_treino

X_train["dificuldade_habilidade"] =
skill_bruta_treino.map(dificuldade_por_habilidade).values
X_test["dificuldade_habilidade"] = (

skill_bruta_teste.map(dificuldade_por_habilidade).fillna(dificuldade_geral_treino).values
)

# Eu atualizo a lista de features para refletir a coluna nova
FEATURES = list(X_train.columns)

n_sem_correspondencia =
(~skill_bruta_teste.isin(dificuldade_por_habilidade.index)).sum()
print("Dificuldade media por habilidade (so treino):")
print(dificuldade_por_habilidade.sort_values(ascending=False))
print(f"\nLinhas de teste sem habilidade correspondente no treino (usaram o
valor de reserva): {n_sem_correspondencia}")
print(f"\nTotal de features apos a codificacao: {len(FEATURES)}")
```

```

Dificuldade media por habilidade (so treino):
skill_agrupada
Addition and Subtraction Integers          0.404378
Pythagorean Theorem                        0.371950
Conversion of Fraction Decimals Percents  0.358010
Subtraction Whole Numbers                  0.352238
outra                                       0.343844
Proportion                                 0.341843
Addition and Subtraction Fractions         0.322997
Equation Solving Two or Fewer Steps        0.312761
Table                                       0.257616
Equation Solving More Than Two Steps       0.241348
Multiplication and Division Integers       0.221710
Name: acerto, dtype: float64

```

Linhas de teste sem habilidade correspondente no treino (usaram o valor de reserva): 0

Total de features apos a codificacao: 27

Interpretação

As médias de dificuldade calculadas só com o treino variam de 0,22 ("Multiplication and Division Integers", a habilidade mais fácil em média) a 0,40 ("Addition and Subtraction Integers", a mais difícil) — uma amplitude real de quase o dobro entre habilidades, confirmando que existe, de fato, um componente de dificuldade populacional distinto do desempenho individual do aluno. Nenhuma linha do conjunto de teste ficou sem correspondência no treino, então o valor de reserva não precisou ser usado nesta divisão específica — mas o código permanece preparado para o caso, o que importa para robustez caso a divisão treino/teste mude (por exemplo, numa nova validação cruzada com outra semente aleatória).

Linha de base

Antes de treinar o XGBoost, estabeleço uma linha de base simples — um classificador que sempre prevê a classe mais frequente. Um modelo mais complexo só é justificável se superar essa referência trivial de forma clara; apresentar apenas a acurácia do XGBoost, sem comparação, não permite avaliar se o modelo está aprendendo algo real ou apenas refletindo o desbalanceamento da base (o mesmo princípio metodológico já aplicado na comparação de classificadores da análise de texto).

```

In [9]: # Eu crio a linha de base: sempre preve a classe mais frequente, ignorando as
        # features
        linha_base = DummyClassifier(strategy="most_frequent", random_state=42)
        linha_base.fit(X_train, y_train)
        acuracia_linha_base = linha_base.score(X_test, y_test)

        # Eu crio o classificador XGBoost com hiperparametros conservadores
        # (profundidade limitada, para reduzir risco de sobreajuste nesta base)
        modelo_xgb = XGBClassifier(
            n_estimators=200,
            max_depth=4,
            learning_rate=0.1,
            eval_metric="logloss",

```

```

    random_state=42,
)
modelo_xgb.fit(X_train, y_train)
acuracia_xgb_teste = modelo_xgb.score(X_test, y_test)

print(f"Acuracia da linha de base (classe majoritaria):
{acuracia_linha_base:.3f}")
print(f"Acuracia do XGBoost (conjunto de teste): {acuracia_xgb_teste:.3f}")

```

Acuracia da linha de base (classe majoritaria): 0.661

Acuracia do XGBoost (conjunto de teste): 0.726

Validação cruzada estratificada e agrupada por aluno

Um único split treino/teste pode ser favorável ou desfavorável por acaso. Uso validação cruzada com 5 dobras sobre o conjunto de treino, para o XGBoost e para a linha de base — com o mesmo `StratifiedGroupKFold` já usado na divisão treino/teste, para que nenhuma dobra de validação misture interações do mesmo aluno com o restante do treino daquela dobra.

```

In [10]: escores_xgb = cross_val_score(
    modelo_xgb, X_train, y_train, groups=grupos_train,
    cv=validacao_estratificada, scoring="accuracy",
)
escores_linha_base = cross_val_score(
    linha_base, X_train, y_train, groups=grupos_train,
    cv=validacao_estratificada, scoring="accuracy",
)

print(f"XGBoost – acuracia media na validacao cruzada: {escores_xgb.mean():.3f}
(desvio padrao {escores_xgb.std():.3f})")
print(f"Linha de base – acuracia media na validacao cruzada:
{escores_linha_base.mean():.3f} (desvio padrao
{escores_linha_base.std():.3f})")

```

XGBoost – acuracia media na validacao cruzada: 0.725 (desvio padrao 0.004)

Linha de base – acuracia media na validacao cruzada: 0.664 (desvio padrao 0.008)

Interpretação

O XGBoost supera a linha de base em todas as dobras da validação cruzada agrupada por aluno (0,725 contra 0,664 de acurácia média, um ganho de 6 pontos percentuais), com desvio padrão baixo (0,004). Esse resultado é o melhor entre as quatro rodadas de desenvolvimento (0,699 → 0,713 → 0,723 → 0,725) — cada rodada de engenharia de features produziu um ganho real, ainda que decrescente: o salto da primeira para a segunda rodada foi de 1,4 ponto percentual, da segunda para a terceira foi de 1,0 ponto, e desta rodada foi de apenas 0,2 ponto. Esse padrão de retorno decrescente é esperado e coerente com a decisão de encerrar a busca por novas features após esta rodada: o espaço de sinal disponível neste dataset específico (sem timestamp, sem dado de latência) está próximo do limite prático de exploração.

Fase 5 — Avaliação

Além da acurácia, reporto precisão, revocação e F1 por classe no conjunto de teste — a acurácia isolada pode mascarar um desempenho ruim na classe minoritária (erro), que é justamente a classe de maior interesse pedagógico: é o erro previsto com antecedência que aciona a adaptação da atividade.

```
In [11]: from sklearn.metrics import classification_report, confusion_matrix

previsoes_teste = modelo_xgb.predict(X_test)

print("Relatorio de classificacao (XGBoost, conjunto de teste):")
print(classification_report(y_test, previsoes_teste, target_names=["erro",
"acerto"]))

print("Matriz de confusao:")
print(confusion_matrix(y_test, previsoes_teste))
```

Relatorio de classificacao (XGBoost, conjunto de teste):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

erro	0.68	0.36	0.47	13269
acerto	0.74	0.91	0.81	25904
accuracy			0.73	39173
macro avg	0.71	0.64	0.64	39173
weighted avg	0.72	0.73	0.70	39173

Matriz de confusao:

```
[[ 4827  8442]
 [ 2287 23617]]
```

Interpretação

A métrica agregada de acurácia esconde uma assimetria importante: o recall da classe "erro" é de 0,36 — o melhor entre as quatro rodadas (0,25 → 0,29 → 0,35 → 0,36), mas ainda insuficiente para o objetivo de negócio. Em contraste, o recall da classe "acerto" é 0,91. Essa assimetria é esperada dado o desbalanceamento da base (66,3% de acertos) e do limiar de decisão padrão (0,5): como acertar a classe majoritária já rende acurácia alta, o modelo tem pouco incentivo, na otimização padrão, a arriscar prever a classe minoritária. O objetivo de negócio definido na Fase 1 — sinalizar risco de erro com antecedência suficiente para adaptar a atividade — ainda não está bem atendido por este modelo na sua forma padrão, otimizada para acurácia geral. Como o custo de deixar passar um aluno em risco é maior do que o custo de um alarme falso ocasional, a seção seguinte corrige o modelo para priorizar recall da classe de risco de forma deliberada.

Fase 4 (revisitada) — Corrigindo o modelo para priorizar recall

Por que recall importa mais que acurácia neste problema

O erro de um classificador binário se divide em dois tipos, e eles não têm o mesmo custo aqui. Um **falso positivo** (o sistema sinaliza risco e o aluno ia bem) custa pouco: na pior das hipóteses, uma pausa ou uma versão mais simples da atividade é oferecida sem necessidade estrita. Um **falso negativo** (o sistema não sinaliza e o aluno estava em risco) custa mais: é exatamente a janela de intervenção que o sistema deveria abrir, fechando sem que ninguém tenha percebido. Um classificador otimizado para acurácia geral, como o da seção anterior, não distingue esses dois custos — ele só minimiza o número total de previsões erradas, e como a classe "acerto" é maioria, a estratégia de menor risco para a métrica de acurácia é justamente ser conservador demais ao prever risco. As duas técnicas a seguir corrigem essa distorção: peso de classe no treinamento e ajuste do limiar de decisão.

Para deixar o objetivo explícito no próprio alvo do modelo, redefino a variável-alvo como "risco" (1 quando o aluno erra a tentativa, 0 quando acerta) — o inverso da variável `acerto` usada até aqui, mas semanticamente mais direta para o que se quer detectar.

Peso de classe no treinamento

Informo ao XGBoost, via o parâmetro `scale_pos_weight`, que um erro de classificação na classe de risco deve pesar mais do que um erro na classe sem risco, proporcionalmente ao desbalanceamento entre elas — isso desloca a fronteira de decisão do modelo para ser menos conservador ao prever risco.

```
In [12]: # Eu redefino o alvo como risco (1 = erro, 0 = acerto) para deixar a semantica
          # explicita
          y_train_risco = 1 - y_train
          y_test_risco = 1 - y_test

          # Eu calculo o peso da classe positiva a partir do desbalanceamento real do
          # treino
          n_negativos = (y_train_risco == 0).sum()
          n_positivos = (y_train_risco == 1).sum()
          peso_classe_risco = n_negativos / n_positivos
          print(f"Proporcao acerto:risco no treino = {n_negativos}:{n_positivos} ->
          scale_pos_weight = {peso_classe_risco:.3f}")

          modelo_xgb_ponderado = XGBClassifier(
              n_estimators=200,
              max_depth=4,
              learning_rate=0.1,
              eval_metric="logloss",
              scale_pos_weight=peso_classe_risco,
              random_state=42,
          )
          modelo_xgb_ponderado.fit(X_train, y_train_risco)

          previsoes_ponderadas = modelo_xgb_ponderado.predict(X_test)
          print("\nRelatorio de classificacao (modelo ponderado, limiar padrao 0,5):")
          print(classification_report(y_test_risco, previsoes_ponderadas, target_names=
          ["acerto", "risco"]))
          print("Matriz de confusao:")
          print(confusion_matrix(y_test_risco, previsoes_ponderadas))
```

Proporcao acerto:risco no treino = 143237:72593 -> scale_pos_weight = 1.973

Relatorio de classificacao (modelo ponderado, limiar padrao 0,5):

	precision	recall	f1-score	support
acerto	0.79	0.72	0.75	25904
risco	0.53	0.63	0.58	13269
accuracy			0.69	39173
macro avg	0.66	0.67	0.66	39173
weighted avg	0.70	0.69	0.69	39173

Matriz de confusao:

```
[[18626  7278]
 [ 4975  8294]]
```

O peso de classe muda o comportamento do modelo de forma acentuada mesmo mantendo o limiar padrão (0,5): o recall da classe "risco" salta de 0,36 (modelo original, sem peso) para 0,63 — o melhor resultado neste limiar entre as quatro rodadas de desenvolvimento (0,58 → 0,58 → 0,60 → 0,63). O ganho tem custo: a precisão da classe "risco" fica em 0,53 e a acurácia geral cai para 0,69 (contra 0,73 do modelo original). Essa troca é esperada e, dado o objetivo de negócio definido na Fase 1, aceitável — o modelo original tinha acurácia mais alta só porque evitava prever a classe minoritária quase por completo, o que é exatamente o comportamento que o peso de classe corrige. Ainda assim, 0,63 de recall está abaixo da meta prática (identificar a maior parte dos casos de risco); a próxima etapa ajusta também o limiar de decisão para buscar um recall mais alto.

Comparação com outros algoritmos

Até aqui, o XGBoost só foi comparado contra uma linha de base trivial (a classe majoritária) — isso mostra que ele aprende algo real, mas não mostra que é a melhor escolha entre modelos igualmente razoáveis para este problema. Comparo agora, sob as mesmas condições (mesmas features, mesmo esquema de peso de classe, mesma validação cruzada estratificada de 5 dobras), três algoritmos com vieses estruturais diferentes: Regressão Logística (fronteira linear, referência clássica), Random Forest (conjunto de árvores independentes, sem o boosting sequencial do XGBoost) e o próprio XGBoost. Como o objetivo agora é recall da classe de risco, não acurácia, essa é a métrica usada para decidir entre eles — junto com precisão, F1 e AUC-ROC como métricas de apoio, para não escolher um modelo que otimize recall às custas de todo o resto.

```
In [13]: from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import recall_score, precision_score, f1_score,
roc_auc_score

candidatos = {
    "Regressao Logistica": lambda: LogisticRegression(
        class_weight="balanced", max_iter=3000, random_state=42,
    ),
    "Random Forest": lambda: RandomForestClassifier(
        n_estimators=200, max_depth=8, class_weight="balanced",
        random_state=42, n_jobs=-1,
    ),
    "XGBoost": lambda: XGBClassifier(
```

```

        n_estimators=200, max_depth=4, learning_rate=0.1,
eval_metric="logloss",
        scale_pos_weight=peso_classe_risco, random_state=42,
    ),
}

# Eu chamo predict()/predict_proba() diretamente sobre cada modelo treinado em
# cada dobra, em vez de usar o scoring automatico do cross_validate – o scorer
# roc_auc do scikit-learn 1.7 consulta uma tag de classificador que o XGBoost
# 2.0.x (versao fixada por compatibilidade com o shap, ver requirements.txt)
# nao expoe corretamente, o que faz o roc_auc vir sempre NaN para esse modelo
# especifico quando calculado pelo caminho automatico
resultados_comparacao = {nome: {"recall": [], "precision": [], "f1": [],
"roc_auc": []} for nome in candidatos}

for indice_treino, indice_validacao in validacao_estratificada.split(X_train,
y_train_risco, groups=grupos_train):
    X_treino_dobra = X_train.iloc[indice_treino]
    X_val_dobra = X_train.iloc[indice_validacao]
    y_treino_dobra = y_train_risco.iloc[indice_treino]
    y_val_dobra = y_train_risco.iloc[indice_validacao]

    for nome_modelo, construtor in candidatos.items():
        modelo = construtor()
        modelo.fit(X_treino_dobra, y_treino_dobra)
        previsoes = modelo.predict(X_val_dobra)
        probabilidades = modelo.predict_proba(X_val_dobra)[: , 1]

        resultados_comparacao[nome_modelo]
["recall"].append(recall_score(y_val_dobra, previsoes))
        resultados_comparacao[nome_modelo]
["precision"].append(precision_score(y_val_dobra, previsoes))
        resultados_comparacao[nome_modelo]["f1"].append(f1_score(y_val_dobra,
previsoes))
        resultados_comparacao[nome_modelo]
["roc_auc"].append(roc_auc_score(y_val_dobra, probabilidades))

tabela_comparacao = pd.DataFrame({
    nome: {metrica: sum(valores) / len(valores) for metrica, valores in
metricas.items()}
    for nome, metricas in resultados_comparacao.items()
}).T

print("Comparacao entre algoritmos (validacao cruzada, 5 dobras, agrupada por
aluno, alvo = risco):")
print(tabela_comparacao.round(3))

```

Comparacao entre algoritmos (validacao cruzada, 5 dobras, agrupada por aluno, alvo = risco):

	recall	precision	f1	roc_auc
Regressao Logistica	0.577	0.521	0.548	0.716
Random Forest	0.596	0.530	0.561	0.729
XGBoost	0.624	0.528	0.572	0.737

Interpretação

Com o conjunto de features completo, o XGBoost se destaca dos outros dois algoritmos com a maior margem observada em todas as rodadas: recall 0,624, precisão 0,528, F1 0,572 e AUC-ROC 0,737 — à frente de Random Forest (0,596 / 0,530 / 0,561 / 0,729) e,

com margem maior ainda, de Regressão Logística (0,577 / 0,521 / 0,548 / 0,716) em quase todas as métricas (a precisão do Random Forest é marginalmente maior). Esse padrão confirma a leitura já registrada na rodada anterior: quanto mais features envolvem interações (sequência de erros combinada com taxa de acerto histórica, contagem de tentativas por habilidade combinada com a taxa naquela habilidade), maior a vantagem de um modelo baseado em árvores sobre um modelo linear, que não capta essas interações sem que elas sejam construídas manualmente como features adicionais. Mantenho o XGBoost como modelo de produção desta análise, agora com a evidência mais forte das quatro rodadas de que essa escolha é a correta para este problema específico.

Ajuste do limiar de decisão

O peso de classe já desloca a fronteira de decisão, mas o limiar de corte da probabilidade continua fixo em 0,5 por padrão — um valor arbitrário do ponto de vista do objetivo de negócio, não algo que precise necessariamente ser mantido. Uso

`precision_recall_curve` para inspecionar, para cada limiar possível, qual seria a precisão e a revocação resultantes, e escolho o menor limiar que atinge uma meta mínima de recall de 0,80 para a classe de risco — abaixo desse limiar, a decisão passa a sinalizar risco com mais frequência, aceitando mais falsos positivos em troca de perder menos alunos em risco real.

```
In [14]: from sklearn.metrics import precision_recall_curve

META_RECALL_MINIMO = 0.80

# Eu obtenho a probabilidade prevista de risco (classe 1) para cada exemplo de teste
probabilidades_risco = modelo_xgb_ponderado.predict_proba(X_test)[: , 1]

precisoes, recalls, limiares = precision_recall_curve(y_test_risco,
probabilidades_risco)

# precision_recall_curve retorna um ponto a mais que limiares (o ultimo ponto
nao tem limiar associado);
# eu descarto esse ultimo ponto para alinhar os tres vetores
precisoes, recalls = precisoes[:-1], recalls[:-1]

# Eu busco o maior limiar que ainda atinge a meta de recall
# (limiares mais altos tendem a reduzir recall e aumentar precisao)
indices_validos = recalls >= META_RECALL_MINIMO
limiar_escolhido = limiares[indices_validos].max()
indice_escolhido = list(limiares).index(limiar_escolhido)

print(f"Limiar escolhido: {limiar_escolhido:.3f}")
print(f"Recall nesse limiar: {recalls[indice_escolhido]:.3f}")
print(f"Precisao nesse limiar: {precisoes[indice_escolhido]:.3f}")

previsoes_limiar_ajustado = (probabilidades_risco >=
limiar_escolhido).astype(int)
print("\nRelatorio de classificacao (modelo ponderado, limiar ajustado):")
print(classification_report(y_test_risco, previsoes_limiar_ajustado,
target_names=["acerto", "risco"]))
```

```
print("Matriz de confusao:")
print(confusion_matrix(y_test_risco, previsoes_limiar_ajustado))
```

Limiar escolhido: 0.388
Recall nesse limiar: 0.800
Precisao nesse limiar: 0.450

Relatorio de classificacao (modelo ponderado, limiar ajustado):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

acerto	0.83	0.50	0.62	25904
risco	0.45	0.80	0.58	13269
accuracy			0.60	39173
macro avg	0.64	0.65	0.60	39173
weighted avg	0.70	0.60	0.61	39173

Matriz de confusao:

```
[[12919 12985]
 [ 2653 10616]]
```

Interpretação

Reduzindo o limiar de decisão de 0,5 para 0,388, o recall da classe "risco" atinge a meta de 0,80. A precisão nesse mesmo recall é 0,45 — a melhor entre as quatro rodadas (0,41 → 0,42 → 0,44 → 0,45) — e a acurácia geral, 0,60, também é a mais alta no mesmo ponto de operação (0,54 → 0,55 → 0,59 → 0,60). O padrão se repete: cada rodada de engenharia de features tornou a troca entre recall e precisão menos custosa, sem nunca abrir mão da meta de recall de 0,80 definida como prioridade de negócio. Essa é exatamente a troca que a Fase 4 revisitada definiu como aceitável: dado que o custo de deixar passar um aluno em risco é maior do que o custo de um alarme falso ocasional, um recall de 0,80 à custa de mais falsos positivos atende melhor ao objetivo de negócio do que a versão original. Em produção, o volume de falsos positivos que ainda resta motivaria usar o sinal do modelo como um filtro de atenção (que atividades merecem revisão do professor), não como uma decisão automática, e cruzá-lo com outros sinais (como o Grafo de Conhecimento do Núcleo 2) para reduzir o ruído.

Fase 6 — Implantação

Explicabilidade com SHAP

Um modelo que prevê risco sem explicar a previsão tem utilidade limitada em um contexto pedagógico — o professor precisa entender *por que* o sistema sinalizou risco para agir sobre a causa, não só sobre o alarme. Uso SHAP (*SHapley Additive exPlanations*) para quantificar a contribuição de cada feature em cada previsão individual, uma técnica baseada em teoria dos jogos que atribui a cada variável uma parcela do afastamento da previsão em relação à média do modelo — o mesmo princípio de explicabilidade citado como requisito de projeto (seção de Ética em IA), aplicado aqui a um modelo de fato treinado, complementando a explicabilidade por regra já usada nas análises anteriores.

A explicação a seguir usa o **modelo ponderado** (`modelo_xgb_ponderado`), o que de fato seria usado em produção após a correção de recall da fase anterior — explicar o modelo descartado (otimizado só para acurácia) não teria utilidade prática.

```
In [15]: import shap

# Eu crio o explicador SHAP especifico para modelos de arvore (mais eficiente
# que o generico),
# agora sobre o modelo ponderado – a classe positiva (1) e' "risco"
explicador = shap.TreeExplainer(modelo_xgb_ponderado)
valores_shap = explicador.shap_values(X_test)

# Eu calculo a importancia media absoluta de cada feature, como um resumo
agregado
importancia_media = pd.Series(
    abs(valores_shap).mean(axis=0), index=FEATURES,
).sort_values(ascending=False)

print("Importancia media (|valor SHAP|) por feature:")
print(importancia_media)
```

```
C:\Users\fredericorosa\AppData\Local\Programs\Python\Python310\lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter
and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

```
Importancia media (|valor SHAP|) por feature:
taxa_acerto_na_habilidade          0.343648
taxa_acerto_historica              0.210777
sequencia_erros_anterior           0.201706
n_tentativas_habilidade_anteriores 0.125053
taxa_acerto_recente                0.110844
choose_1                           0.105780
tentativas_media_recente           0.050455
dificuldade_habilidade             0.045604
posicao                             0.043649
tentativas_media_historica         0.038427
Multiplication and Division Integers 0.025006
Equation Solving Two or Fewer Steps 0.023641
Equation Solving More Than Two Steps 0.023595
algebra                            0.018787
tendencia_desempenho              0.015063
outra                              0.012211
Pythagorean Theorem               0.011526
Proportion                        0.011262
Table                             0.010542
Addition and Subtraction Fractions 0.009796
Conversion of Fraction Decimals Percents 0.004974
Subtraction Whole Numbers          0.004427
volatilidade_recente              0.004055
Addition and Subtraction Integers  0.002525
fill_in_1                         0.001904
open_response                     0.000092
choose_n                          0.000044
dtype: float32
```

Cards explicáveis — linguagem técnica e linguagem acessível

A mesma explicação SHAP precisa de dois registros de linguagem diferentes: o professor lida com terminologia pedagógica e pode interpretar um valor de importância diretamente; o responsável pela criança não tem por que conhecer o vocabulário técnico do modelo. A função abaixo traduz a mesma previsão individual nos dois formatos, a partir dos mesmos valores SHAP — a explicação é a mesma, muda só a linguagem em que ela é apresentada.

```
In [16]: TRADUCAO_FEATURE = {
    "taxa_acerto_historica": "o quanto a criança tem acertado desde o início",
    "taxa_acerto_recente": "o quanto a criança tem acertado nas últimas tentativas",
    "volatilidade_recente": "o quanto o desempenho da criança tem variado (estável ou inconstante)",
    "tendencia_desempenho": "se o desempenho da criança está melhorando ou piorando",
    "tentativas_media_historica": "quantas tentativas a criança costuma precisar",
    "tentativas_media_recente": "quantas tentativas a criança tem precisado recentemente",
    "taxa_acerto_na_habilidade": "o quanto a criança acerta nesse tipo específico de exercício",
    "n_tentativas_habilidade_anteriores": "quantas vezes a criança já praticou esse tipo de exercício",
    "dificuldade_habilidade": "o quão difícil esse tipo de exercício costuma ser para as crianças em geral",
    "sequencia_erros_anterior": "quantos erros seguidos a criança acabou de ter",
    "posicao": "quantos exercícios a criança já fez nesta sessão",
}

def gerar_card_explicativo(indice_exemplo: int) -> dict:
    """Traduz a previsão SHAP de um exemplo em card técnico e card acessível."""
    valores_exemplo = valores_shap[indice_exemplo]
    # classe positiva (índice 1) e' "risco", com o modelo ponderado
    previsao = modelo_xgb_ponderado.predict_proba(X_test.iloc[[indice_exemplo]])[0][1]

    contribuicoes = sorted(
        zip(FEATURES, valores_exemplo), key=lambda item: abs(item[1]),
        reverse=True,
    )
    principal_feature, principal_valor = contribuicoes[0]

    card_tecnico = {
        "probabilidade_risco": round(float(previsao), 3),
        "feature_mais_influente": principal_feature,
        "contribuicao_shap": round(float(principal_valor), 3),
        "todas_contribuicoes": {f: round(float(v), 3) for f, v in contribuicoes},
    }

    direcao = "aumentou" if principal_valor > 0 else "reduziu"
    descricao_feature = TRADUCAO_FEATURE.get(principal_feature, principal_feature)
    card_acessivel = (
        f"Risco de dificuldade nesta atividade: {previsao:.0%}. "
```

```

        f"O fator que mais {direcao} esse risco foi {descricao_feature}."
    )

    return {"tecnico": card_tecnico, "acessivel": card_acessivel}

exemplo = gerar_card_explicativo(0)
print("Card tecnico (professor):")
print(exemplo["tecnico"])
print("\nCard acessivel (responsavel):")
print(exemplo["acessivel"])

```

Card tecnico (professor):

```

{'probabilidade_risco': 0.502, 'feature_mais_influente': 'posicao',
 'contribuicao_shap': -0.164, 'todas_contribuicoes': {'posicao': -0.164,
 'sequencia_erros_anterior': 0.134, 'taxa_acerto_historica': 0.126,
 'n_tentativas_habilidade_anteriores': -0.082, 'tentativas_media_recente': 0.058,
 'taxa_acerto_recente': -0.049, 'choose_1': 0.047, 'volatilidade_recente': -0.035,
 'taxa_acerto_na_habilidade': 0.034, 'tentativas_media_historica': -0.029,
 'Equation Solving Two or Fewer Steps': -0.012, 'Equation Solving More Than Two
 Steps': -0.011, 'Multiplication and Division Integers': 0.01,
 'tendencia_desempenho': -0.01, 'outra': -0.01, 'Table': 0.009,
 'dificuldade_habilidade': -0.008, 'Addition and Subtraction Fractions': 0.004,
 'Proportion': -0.004, 'Conversion of Fraction Decimals Percents': -0.003,
 'Pythagorean Theorem': -0.001, 'algebra': -0.001, 'Addition and Subtraction
 Integers': 0.001, 'Subtraction Whole Numbers': 0.0, 'fill_in_1': 0.0,
 'open_response': -0.0, 'choose_n': 0.0}}

```

Card acessivel (responsavel):

Risco de dificuldade nesta atividade: 50%. O fator que mais reduziu esse risco foi quantos exercícios a criança já fez nesta sessão.

Interpretação

A explicação SHAP valida, com evidência do próprio modelo, cada uma das features acrescentadas nesta rodada — com uma exceção clara.

`n_tentativas_habilidade_anteriores` (importância 0,13) confirma a hipótese de que a confiabilidade da estimativa de `taxa_acerto_na_habilidade` (a feature mais importante do modelo, 0,34) é, ela mesma, informação útil. `posicao` (0,04) e `dificuldade_habilidade` (0,05) têm contribuição moderada, mas real — o exemplo de card mostra `posicao` como a feature mais influente numa previsão específica, reduzindo o risco estimado (aluno mais avançado na sessão, sem sinal de fadiga naquele caso). A exceção é `volatilidade_recente`: importância de apenas 0,004, a segunda mais baixa entre as 27 features do modelo — a hipótese de que a instabilidade do desempenho recente adicionaria sinal além da própria taxa de acerto recente não se confirmou nesta base. Esse é um resultado tão válido quanto os positivos: mantê-la no modelo não prejudica o desempenho (XGBoost lida bem com features de baixo valor preditivo, simplesmente atribuindo pouco peso a elas), mas fica registrado que essa hipótese específica não rendeu o ganho esperado. O card acessível mantém a mesma tradução para linguagem não técnica, agora escolhendo entre um conjunto ainda maior de fatores possíveis.

```
In [17]: # Eu encerro a sessão Spark, liberando os recursos
spark.stop()
```

Síntese do ciclo CRISP-DM

Fase	Resultado
1. Negócio	Previsão de risco de erro na próxima tentativa como proxy comportamental de frustração/fadiga, complementar ao sinal afetivo medido pela pulseira
2. Dados	ASSISTments 2009 (274.331 interações individuais, 4.148 alunos), processado via Spark
3. Preparação	Explosão das listas em interações via <code>arrays_zip / explode</code> ; features derivadas por função de janela — taxa de acerto acumulada, em janela recente e por habilidade específica; tendência; volatilidade recente; sequência de erros consecutivos; contagem de tentativas por habilidade; posição na sessão; tipo de resposta esperado; dificuldade média da habilidade (codificada com segurança contra vazamento, só com estatísticas do treino, na Fase 4) — todas sem vazamento de dado
4. Modelagem	Separação treino/teste e validação cruzada com <code>StratifiedGroupKFold</code> (nenhum aluno aparece em dois lados da divisão); XGBoost comparado contra linha de base e contra Regressão Logística/Random Forest; modelo revisitado com peso de classe (<code>scale_pos_weight</code>) e ajuste de limiar de decisão para priorizar recall da classe de risco
5. Avaliação	Métricas por classe (precisão, revocação, F1) e matriz de confusão, não só acurácia agregada; comparação explícita entre o modelo padrão (limiar 0,5) e o modelo corrigido
6. Implantação	Explicabilidade via SHAP sobre o modelo corrigido, com tradução para dois públicos (card técnico e card acessível) a partir da mesma previsão

Trabalhos futuros

- O dataset usado é uma base pública de outro sistema tutor, não dado real do sistema descrito neste projeto — os números de desempenho aqui validam a viabilidade da técnica, não o desempenho esperado em produção.
- A definição de "risco de frustração" usada é um proxy comportamental (erro na próxima tentativa); uma validação futura poderia cruzar esse sinal com o dado afetivo da pulseira biométrica (BPM/GSR), quando disponível de forma integrada, para verificar se os dois sinais concordam.
- Os hiperparâmetros do XGBoost foram fixados de forma conservadora, sem uma busca sistemática (*grid search* ou *random search*). Diferente do que se via nas rodadas anteriores (algoritmos empatados, sinal concentrado nas features), a partir da terceira rodada de features o XGBoost passou a se destacar com folga dos outros dois algoritmos — evidência de que o modelo já está explorando interações relevantes entre features, o que torna uma busca de hiperparâmetros mais promissora agora do que antes. Fica como item de maior potencial entre os pendentes, deliberadamente não executado nesta versão (ver decisão de encerramento abaixo).

- A meta de recall mínimo (0,80) usada para escolher o limiar de decisão foi fixada de forma arbitrária, sem uma análise formal de custo-benefício entre falso positivo e falso negativo neste contexto pedagógico específico — uma escolha mais rigorosa exigiria estimar, junto à equipe pedagógica, o custo relativo real de cada tipo de erro, algo que só é possível com uso em produção.
- O tamanho da janela recente (5 tentativas) foi escolhido por convenção, não por busca sistemática; testar outros tamanhos de janela poderia revelar um ponto ótimo entre capturar deterioração recente e manter uma estimativa estatisticamente estável.
- As features permanecem limitadas ao que o log do ASSISTments oferece: não há timestamp entre tentativas, o que impede qualquer feature de latência (tempo gasto por tentativa, tempo entre uma tentativa e a próxima) — um sinal frequentemente citado na literatura de sistemas tutores inteligentes como forte indicador de frustração, hoje só possível com telemetria real do sistema (`TelemetryEvent` , ver seção 7 do documento de arquitetura do projeto).

Decisão de encerrar a engenharia de features nesta versão

Quatro rodadas de engenharia de features produziram ganho real e mensurável em cada etapa, mas com retorno visivelmente decrescente (1,4 → 1,0 → 0,2 pontos percentuais de acurácia entre rodadas sucessivas). Esse padrão, mais o fato de que o dataset não oferece nenhum sinal de tempo entre tentativas (item já registrado acima), foi o critério usado para decidir explicitamente parar por aqui, em vez de continuar buscando novas features de forma incremental: o espaço de sinal facilmente extraível destas colunas específicas está perto do limite prático. A busca de hiperparâmetros do XGBoost e a busca pelo tamanho ótimo da janela recente continuam sendo os itens de maior potencial ainda não executados — registrados como trabalho futuro por escolha deliberada da equipe, não por esquecimento.